

CUDA Lab 5

Dries Van de Velde, Behzad Iqbal

March 2024

1 Introduction

In this lab, we explored methods to optimise GPU programs for speed by focusing on the distinction between synchronous and asynchronous GPU execution. Traditional synchronous execution involves the CPU waiting for GPU tasks to finish before proceeding with other CPU operations (kernels calls are async by default). However, asynchronous execution allows the CPU to continue with other tasks while the GPU performs its computations. This asynchronous approach can lead to significant speed improvements due to it being some form of parallelisation.

2 Exercise

In this exercise, we revisited the previous striding GPU kernel and created four versions: one for finding the maximum, one for the minimum, one for the product, and one for the summation of an array. We compared the execution times of both synchronous and asynchronous versions. The primary goal of the asynchronous approach was to maximise CPU utilisation while the GPU was busy. This involved overlapping CPU tasks such as data preparation and initialisation with GPU computations to minimise overall execution time.

However, in our experiments, the asynchronous approach did not outperform the synchronous approach in any way (see Figure 1). This is primarily because the CPU tasks take considerably longer than GPU computations in each iteration. Thus, the potential benefits of asynchronous execution were limited.

We hypothesise that more substantial differences between synchronous and asynchronous approaches could be observed with increased GPU computation complexity. When the execution times of CPU and GPU tasks become comparable, the asynchronous approach could yield greater improvements. We tested this hypothesis by increasing the complexity of the GPU function, and the results (see Figure 2) supported our expectations. However the async approach still isn't always faster in our tests.

3 Conclusion

In conclusion, while the asynchronous approach to GPU execution shows promise for optimising performance, its effectiveness depends on various factors such as the balance between CPU and GPU task times and the complexity of GPU computations.

Our entire timing results can be found together with our code in our Github Repository.

4 Figures

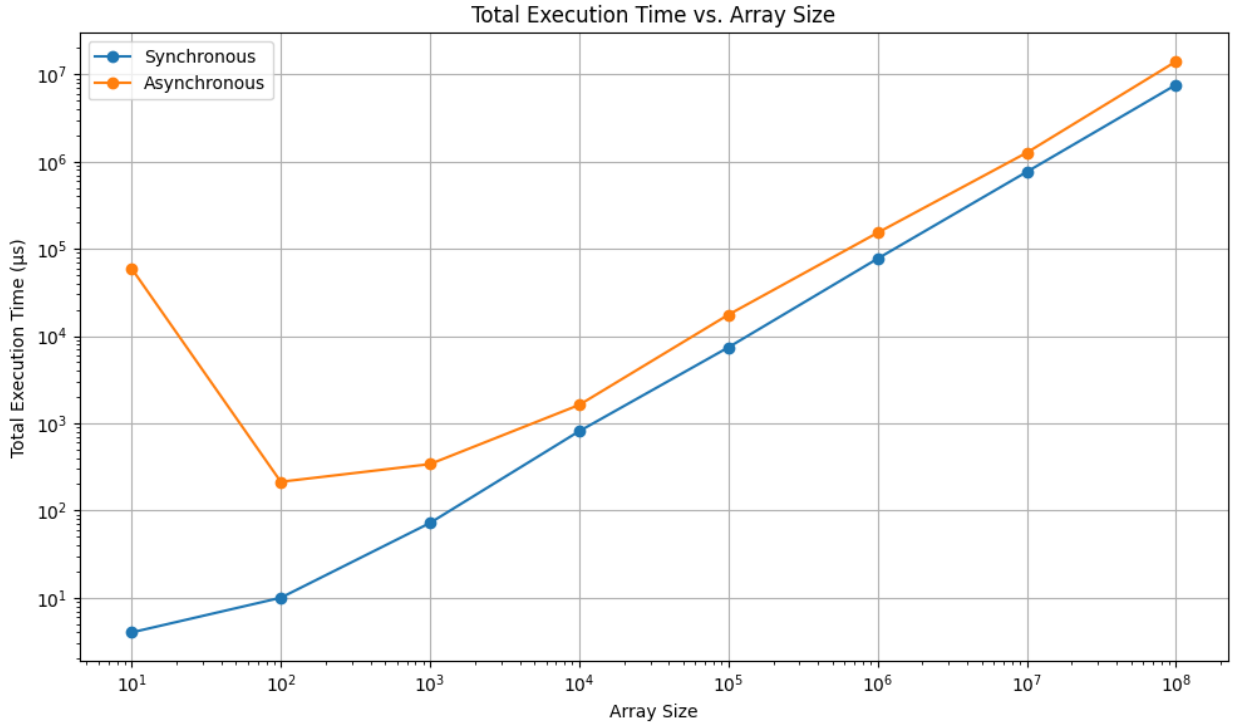


Figure 1: Execution Times vs Matrix Sizes (log scale)

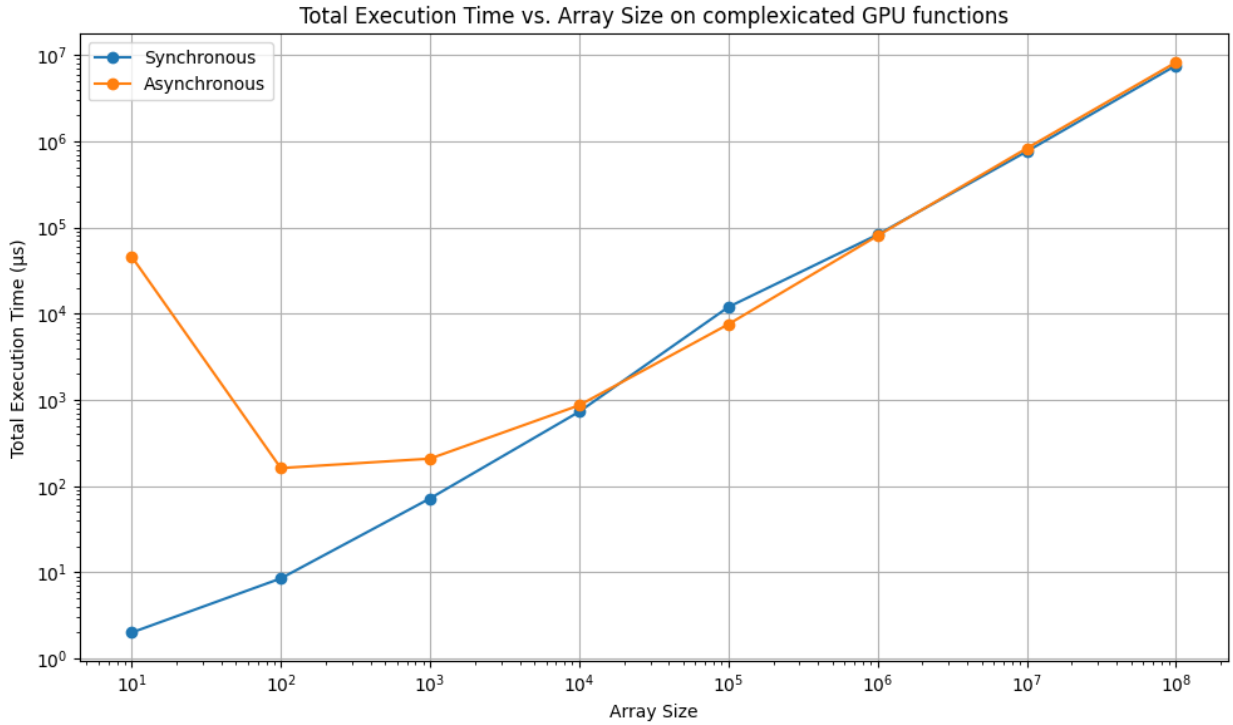


Figure 2: Execution Times vs Matrix Sizes (log scale) with increased complexity